Universidad de San Carlos de Guatemala
Escuela de Ingeniería en Ciencias y Sistemas, Facultad de Ingeniería
Introducción a la programación y computación 2, 2do. Semestre 2026

# API PARA AUTORIZAR DOCUMENTOS TRIBUTARIOS ELECTRÓNICOS EN C#

**202210198 – William René Toledo Corado · Sección B**

**Resumen**

El presente proyecto desarrolla un simulador de autorización de documentos tributarios electrónicos mediante una API HTTP implementada en C# y una aplicación web construida con Razor Pages. La propuesta aplica programación orientada a objetos para separar la presentación, las reglas de negocio y la persistencia en archivos JSON. El sistema administra el Registro Tributario Unificado, valida identificadores NIT, fechas, referencias y montos, y asigna códigos únicos con un correlativo que se reinicia diariamente. Asimismo, permite consultar facturas emitidas y recibidas, resumir movimientos de IVA, visualizar gráficas y exportar reportes PDF. La evaluación utiliza los archivos originales proporcionados para el proyecto y comprueba resultados, concurrencia, reinicios y manejo de entradas inválidas. Las setenta y cuatro comprobaciones funcionales resultaron correctas. Se concluye que la separación de responsabilidades y la confirmación del estado antes de responder facilitan una autorización verificable y reproducible. El alcance corresponde a una demostración académica local, sin conexión con servicios fiscales reales ni almacenamiento distribuido.

**Palabras clave**

API HTTP; C#; POO; JSON; DTE

***Abstract***

*This project develops an electronic tax document authorization simulator through an HTTP API implemented in C# and a web application built with Razor Pages. The proposal applies object-oriented programming to separate presentation, business rules, and persistence in JSON files. The system manages the Unified Tax Registry, validates taxpayer identifiers, dates, references, and monetary amounts, and assigns unique codes with a sequence that restarts daily. It also supports queries for issued and received invoices, summarizes VAT transactions, displays charts, and exports PDF reports. The evaluation uses the original files supplied for the project and checks expected results, concurrency, process restarts, and invalid input handling. All seventy-four functional checks passed. The analysis concludes that separating responsibilities and confirming the stored state before responding support a verifiable and reproducible authorization process. The scope is an academic demonstration running locally, without connections to actual tax services or distributed storage. These limits define the conditions under which the observed results can be reproduced.*

***Keywords***

*HTTP API; C#; OOP; JSON; ETD*

## Introducción

La autorización de una factura electrónica exige comprobar la identidad de sus participantes, la coherencia de sus montos y la unicidad del identificador emitido. El Proyecto 3 de IPC2 plantea integrar estas verificaciones en una API HTTP desarrollada en C# y consumirla mediante Razor Pages (Universidad de San Carlos de Guatemala, 2026). Este ensayo sostiene que separar presentación, reglas de negocio y persistencia facilita explicar y comprobar cada autorización. Se analiza el diseño de las clases, los algoritmos de validación, el control del correlativo diario y la construcción de consultas. Finalmente, se contrastan los resultados con los archivos originales de prueba para discutir el alcance y las limitaciones de la solución.

## Desarrollo del tema

### a. Diseño y comunicación HTTP

El flujo comienza con la actualización del RTU. El cliente envía un arreglo de contribuyentes y la API diferencia las incorporaciones, las actualizaciones y los identificadores inválidos. Luego recibe los lotes de facturas y conserva la información necesaria para explicar cada aprobación o rechazo. Las consultas posteriores se calculan a partir de ese estado guardado. Esta secuencia evita autorizar a participantes que todavía no existen en el registro y permite repetir el proceso con archivos distintos sin cambiar los algoritmos. La interfaz muestra las respuestas JSON de cada archivo para que el usuario pueda contrastar los contadores con sus entradas.

La solución contiene tres proyectos con responsabilidades diferenciadas. SatDte.Web implementa la presentación con Razor Pages; SatDte.Api publica las operaciones HTTP; SatDte.Core reúne las entidades, validaciones, servicios, consultas y mecanismos de persistencia. Razor Pages relaciona cada pantalla con su PageModel, donde se reciben las entradas y se solicita información al servicio (Anderson et al., s. f.). El frontend utiliza HttpClient y no lee el archivo de datos. Esta decisión reproduce la arquitectura del enunciado y permite verificar la API desde Postman sin depender de la interfaz gráfica.

Las entidades principales son Contribuyente y Factura. La primera relaciona el NIT con el nombre vigente del RTU. La segunda conserva los participantes, la fecha documental, los montos, el momento de recepción, los errores encontrados y el código de aprobación cuando existe. EstadoSistema agrupa los contribuyentes, los intentos de facturación, los descartes y los correlativos diarios. Un diccionario facilita la búsqueda de contribuyentes por NIT; una lista conserva los intentos en orden de procesamiento; un conjunto temporal permite detectar referencias autorizadas sin recorrer todas las facturas por cada comparación.

Las abstracciones IRepositorio e IReloj reducen la dependencia de los servicios respecto de detalles externos. ServicioDte solicita al repositorio una operación de lectura o actualización, sin administrar directamente los archivos. ServicioConsultas obtiene una instantánea y construye las agrupaciones necesarias para cada petición. IReloj proporciona el momento actual; RelojGuatemala lo expresa con desplazamiento UTC-6. Durante las pruebas se utiliza un reloj controlado que permite simular el paso de medianoche y comprobar el reinicio del correlativo sin esperar al día siguiente. La encapsulación protege estas decisiones detrás de operaciones explícitas.

Las rutas HTTP son una capa de adaptación al dominio. Reciben JSON o parámetros, invocan los

servicios y devuelven JSON, SVG o PDF. ASP.NET Core permite declarar estos endpoints mediante Minimal APIs (Microsoft, s. f.-a). En este proyecto, las reglas no se concentran en las rutas: permanecen en clases de servicio que también se prueban directamente. Las respuestas conservan las claves exigidas para RTU y el contenedor resultadoAutorizaciones para facturación. Los contadores adicionales y los detalles explican los errores requeridos por los casos de prueba sin cambiar los campos obligatorios del contrato.

El procesamiento distingue errores de la solicitud completa y errores de una fila. Un JSON malformado, un objeto suelto, un arreglo vacío o un lote mixto reciben una respuesta 400 antes de modificar el estado. Una factura correctamente estructurada puede fallar en varias reglas de negocio y se registra como un intento rechazado. La Web muestra la respuesta de cada archivo, mantiene el orden de envío y puede continuar con los archivos siguientes. Así, la interacción ayuda a localizar el problema sin ocultar la respuesta original del servicio.

**b. Validación de datos**

El NIT se representa como texto, porque una base de veinte dígitos puede superar los límites de un entero y porque los ceros iniciales deben conservarse. Una expresión regular verifica que existan entre uno y veinte dígitos de base seguidos de un verificador numérico o K. Se normaliza la letra minúscula para aceptar el caso de borde incluido en el ZIP. El algoritmo separa el verificador, recorre la base de derecha a izquierda y multiplica sus dígitos por pesos que comienzan en dos. La suma se transforma mediante módulo once; el resultado diez equivale a K. Después se compara con el carácter recibido. Esta comprobación precede a la búsqueda en el RTU. Un NIT que no supera el verificador produce un error de identidad inválida. Si lo supera y no aparece en el registro, produce un error de contribuyente inexistente. Aplicar ese orden evita contar el mismo problema en ambas categorías. El ejemplo 5123456 del documento no supera el algoritmo; el sistema no incorpora una excepción para aceptarlo. Las pruebas proporcionadas incluyen ese identificador como inválido y utilizan 5123453 como su contraparte válida. El criterio aplicado queda documentado para facilitar su revisión frente a cualquier aclaración del curso.

La fecha se obtiene de TIEMPO mediante una expresión regular con grupos para día, mes, año, hora y minuto. La expresión localiza la información relevante dentro del texto y descarta nombres de lugares o el sufijo decorativo. Después, DateTime verifica que la combinación exista en el calendario. Esta segunda etapa rechaza, por ejemplo, el 29 de febrero de 2025 y acepta el de 2024. También rechaza las 24:00 y las fechas sin hora. La tolerancia de uno o dos dígitos en día, mes y hora responde a los casos de borde suministrados; no convierte una fecha imposible en una fecha válida.

Para los montos se utiliza decimal, adecuado cuando interesa la precisión de las posiciones decimales, como ocurre con cantidades monetarias (Microsoft, s. f.-b). El IVA esperado es el producto del valor por 0.12, redondeado a dos decimales con MidpointRounding.AwayFromZero. El total se compara con el valor más el IVA recibido. Esta distinción explica por qué una factura con valor 100, IVA 13 y total 113 registra error de IVA, pero no un segundo error de suma. Los valores no numéricos, negativos o con fracciones adicionales se clasifican

como problemas de formato antes de evaluar las reglas tributarias.

La referencia tiene de uno a cuarenta caracteres y se compara exactamente. Una autorización reserva la referencia; una fila rechazada permite corregirla y reenviarla. Dentro de un mismo lote, una referencia aprobada se incorpora inmediatamente al conjunto de referencias utilizadas. La validación acumula los errores de cada factura, en vez de detenerse en el primero. De esta manera, el emisor puede corregir todos los problemas detectados y las estadísticas mantienen contadores independientes sin sumar una factura varias veces al total de rechazadas.

**c. Persistencia y correlativos**

La persistencia debe garantizar que una respuesta de aprobación corresponda a un estado guardado. RepositorioJson mantiene una copia del estado confirmado y ejecuta cada actualización sobre una copia de trabajo. Dentro de un bloqueo, el servicio aplica las verificaciones, registra los intentos y asigna los códigos. Después, el repositorio serializa la copia a un archivo temporal, vacía su contenido a disco y reemplaza el archivo definitivo. Solo al completar esa escritura se adopta el nuevo estado en memoria. Si la operación falla antes de confirmar, la respuesta de éxito no se emite y la copia previa continúa disponible.

El bloqueo cubre la operación completa, incluida la asignación del correlativo. Sin esa exclusión, dos solicitudes podrían leer el mismo último valor y emitir códigos repetidos. El estado conserva un contador por día; la fecha de autorización en Guatemala selecciona el contador que se incrementa. El código combina ocho posiciones de fecha con ocho posiciones de correlativo. Los rechazos y descartes no consumen números. Un archivo abierto con FileShare.None impide que otra instancia del repositorio escriba sobre el mismo estado. La propuesta contempla un proceso API por archivo, no una infraestructura con escritores distribuidos.

Al reiniciar, la API carga el archivo existente y continúa sus contadores. Un archivo ilegible o de versión incompatible detiene el arranque; no se sustituye silenciosamente por un estado vacío. Esta conducta permite distinguir una instalación sin datos de un problema de integridad que requiere revisión. La operación Inicializar es explícita y elimina RTU, facturas, descartes y correlativos. La pantalla muestra una confirmación antes de enviarla, ya que devolver el simulador al estado inicial impide reconstruir sus datos a partir del archivo activo. Para repetir las pruebas se deben cargar de nuevo los originales.

Las consultas obtienen una instantánea que no cambia mientras se calcula el reporte. Para un contribuyente se filtran las autorizadas por NIT, sentido y rango inclusivo; después se agrupan por fecha y se calculan cantidades y montos. El resumen de IVA reúne los NIT que participaron en un día y suma por separado sus movimientos emitidos y recibidos. En el rango general se selecciona el valor sin IVA o el total de cada factura. El reporte de errores incluye los intentos rechazados y cuenta las categorías de cada día, junto con emisores y receptores distintos.

Los descartes de formato se conservan aparte porque no siempre proporcionan una fecha documental utilizable. Se informa su posición, referencia, motivo y momento de recepción, pero no se les inventa una fecha para incorporarlos al reporte diario. Las gráficas SVG presentan las mismas agrupaciones que las tablas. Los PDF se construyen en C# con texto seleccionable y paginación, sin servicios externos. La reutilización de los resultados de consulta para JSON,

gráfica y PDF mantiene un único criterio de cálculo y permite contrastar la pantalla con la información devuelta directamente por HTTP.

La fecha del documento y la fecha de autorización cumplen funciones diferentes. La primera determina el período de las consultas; la segunda selecciona el contador diario utilizado para emitir el código. Por esta razón una factura histórica puede recibir un número generado en la fecha actual sin alterar su mensaje de entrada. Esta separación también permite explicar los resultados cuando se repiten archivos de prueba de años anteriores. El criterio se mantiene tanto en las respuestas de la API como en la información presentada por la Web.

**d. Evaluación del sistema**

La evaluación utiliza los archivos del ZIP proporcionado sin modificarlos y contrasta sus resultados con el LEEME. El primer mensaje del RTU produce nueve incorporaciones, una actualización y tres NIT inválidos. Las facturas de marzo generan cuatro autorizaciones. El conjunto de errores genera tres aprobaciones, diez facturas rechazadas y un descarte de formato; las categorías se verifican de manera independiente. Los casos de borde producen once aprobaciones y dos descartes. La carga de volumen produce cien autorizaciones adicionales. Estos resultados permiten comprobar el comportamiento solicitado con datos que no se originan en los servicios de la solución.

Las consultas también se contrastan con cantidades concretas. Para el NIT 999999K, las emitidas del primero al dos de marzo de 2025 totalizan Q 9,672.32 y las recibidas Q 9,258.06. El rango general alcanza Q 18,930.38 con IVA y Q 16,902.13 sin IVA. En el resumen del dos de marzo, ese contribuyente presenta IVA emitido de Q 494.06 y recibido de Q 392.20. La matriz del reporte de errores se compara completa con las once columnas del LEEME. Esto verifica las agrupaciones y las sumas, además de comprobar que la API acepta los mensajes.

Las pruebas del núcleo incluyen concurrencia, referencias compartidas, cambio de día, reapertura del repositorio y reinicialización. Las pruebas HTTP agregan JSON malformado, códigos de estado, descargas PDF, SVG y un reinicio real del proceso. La ejecución registrada obtuvo veinticuatro comprobaciones correctas del núcleo, treinta comprobaciones HTTP correctas y veinte comprobaciones del frontend correctas. La evaluación de integración verificó la carga múltiple desde Razor, el envío secuencial, las consultas y la inicialización desde la Web. La evidencia se incluye separada del ensayo para permitir su reproducción.

La separación de responsabilidades permite relacionar un resultado observable con una regla y una prueba. El diseño orientado a objetos encapsula la persistencia y el reloj; las expresiones regulares extraen la información relevante; el uso de decimal mantiene un criterio monetario explícito. La escritura transaccional y la exclusión de solicitudes simultáneas sostienen la unicidad de los códigos dentro de las condiciones locales de ejecución. La solución ofrece, por tanto, un flujo completo de registro, validación, autorización y análisis, con respuestas que explican tanto las aprobaciones como los errores.

## Conclusiones

La solución integra registro, validación, autorización y consulta mediante una arquitectura orientada a objetos. Las interfaces del repositorio y del reloj permiten comprobar reglas del dominio sin depender de la pantalla ni del momento real de ejecución. El

frontend Razor consume las mismas operaciones HTTP que puede utilizar otro cliente; esa separación hace posible verificar los resultados por más de una vía.

La combinación de validación previa, cálculo monetario con decimal y control del correlativo produce respuestas explicables. Las pruebas con entradas originales, solicitudes concurrentes y reinicios confirman los resultados dentro de las condiciones evaluadas. Guardar el estado antes de emitir una aprobación evita presentar como confirmada una modificación que todavía no ha sido escrita. Además, distinguir los errores por categoría permite corregir una factura sin perder la información de los intentos anteriores.

El almacenamiento local y el único proceso escritor constituyen límites del diseño. La demostración no sustituye una certificación tributaria real ni comprueba operación distribuida. Para ampliar su alcance habría que evaluar nuevos mecanismos de concurrencia, alojamiento y recuperación. Los resultados obtenidos respaldan el propósito académico del proyecto: relacionar una regla, su implementación y una evidencia reproducible de funcionamiento.

## Referencias bibliográficas

Anderson, R., Brock, D., & Larkin, K. (s. f.). *Razor Pages architecture and concepts in ASP.NET Core*. Microsoft Learn. https://learn.microsoft.com/en-us/aspnet/core/razor-pages/?view=aspnetcore-10.0

Microsoft (s. f.-a). *APIs overview*. Microsoft Learn. https://learn.microsoft.com/en-us/aspnet/core/fundamentals/apis?view=aspnetcore-10.0

Microsoft (s. f.-b). *Floating-point numeric types (C# reference)*. Microsoft Learn. https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/builtin-types/floating-point-numeric-types

Universidad de San Carlos de Guatemala (2026). *Proyecto 3 de Introducción a la Programación y Computación 2*. Facultad de Ingeniería. Enunciado y archivos de prueba del segundo semestre.

## Anexo A Modelo de clases

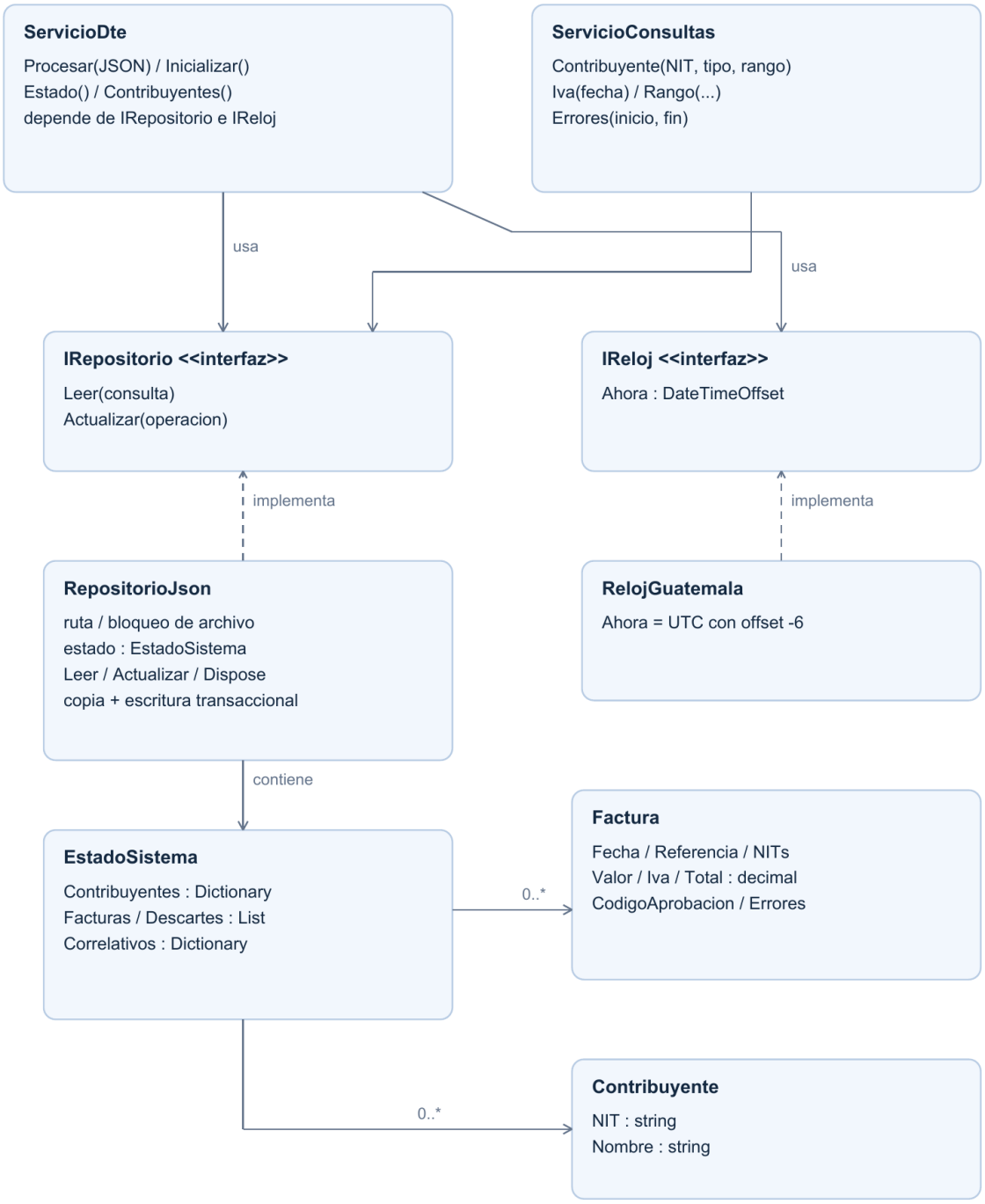



Figura 1. Clases e interfaces principales y sus relaciones en el núcleo del sistema.

Fuente: elaboración propia.

## Anexo B Actividad de procesamiento

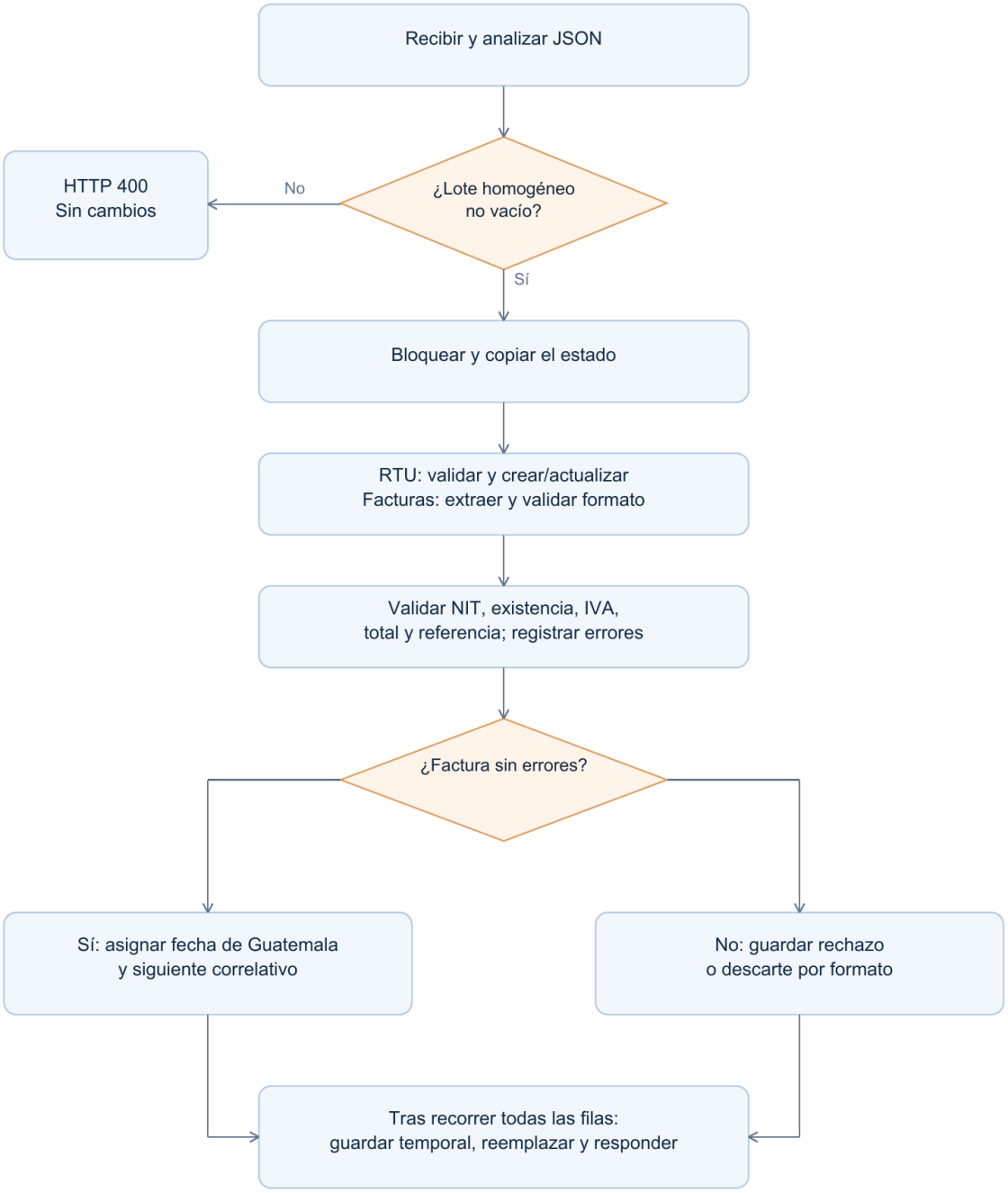



Figura 2. Recepción, validación, autorización y confirmación de un lote. Los descartes omiten las reglas de negocio; los mensajes RTU solo crean o actualizan contribuyentes válidos.

Fuente: elaboración propia.

## Anexo C Validación y consultas

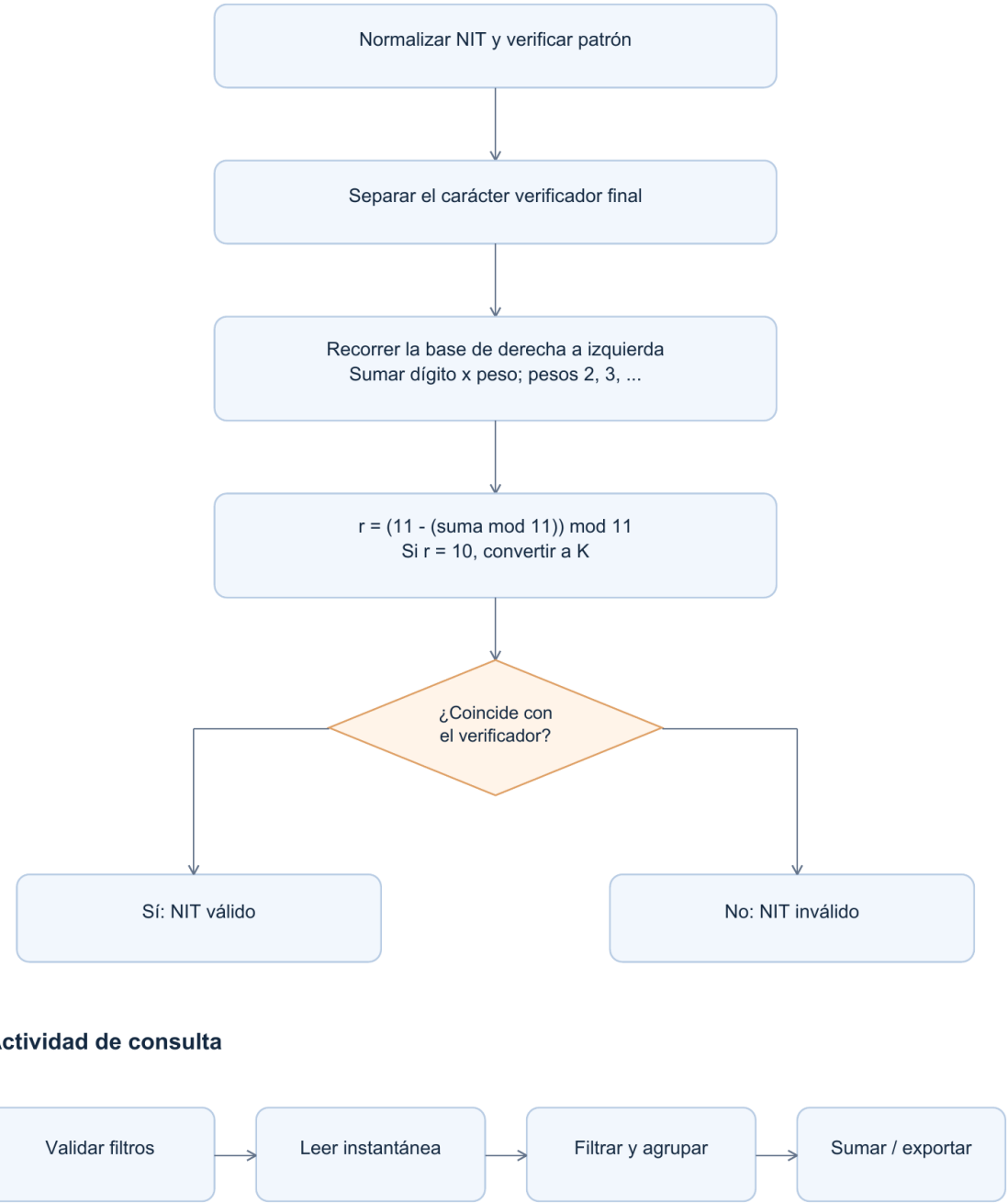



Figura 3. Validación del NIT mediante módulo once y secuencia de construcción de consultas.

Fuente: elaboración propia.

## Anexo D Despliegue del sistema

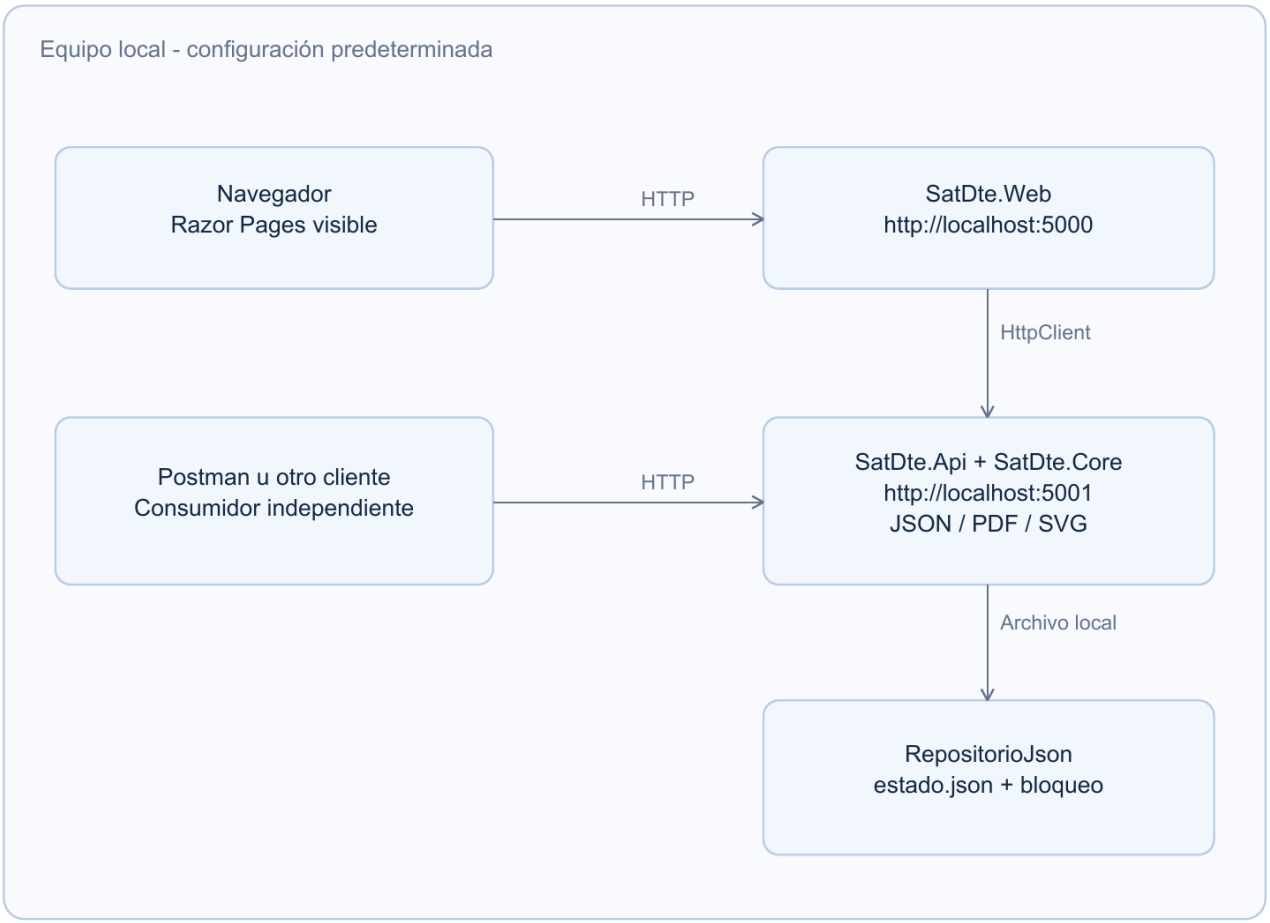



Figura 4. Despliegue local de Razor Pages, API HTTP y repositorio JSON.

Fuente: elaboración propia.